

LAB-1

PostgreSQL 18.6 Native Physical Streaming Replication

AWS EC2 | Amazon Linux 2023 | DB1 Primary → DB2 Hot Standby

Document Type	Technical Standard Operating Procedure (SOP)
Lab Status	Completed and validated
PostgreSQL	18.6
Platform	Amazon EC2 / Amazon Linux 2023
Replication	Asynchronous physical streaming replication

Lab credentials are intentionally included because this document is for a non-production training environment.

Table of Contents

Section	Page
1. Purpose and Scope	3
2. Final Lab Architecture	3
3. Environment Inventory	3
4. Prerequisites	4
5. Create the AWS EC2 Lab Servers	5
6. Configure Security Group and Inbound Rules	5
7. Connect to DB1 and DB2 by SSH	6
8. Configure Hostnames and Private Name Resolution	6
9. Install PostgreSQL 18.6 on Both Servers	7
10. Initialize and Start PostgreSQL	7
11. Configure PostgreSQL Network Listening	8
12. Configure pg_hba.conf and Validate Node Connectivity	8
13. Create the Lab Database and Test Login	9
14. Prepare DB1 as the Replication Primary	9
15. Convert DB2 into the Physical Standby	11
16. Start and Validate Streaming Replication	12
17. End-to-End Data Replication Validation	12
18. Replication Health-Check Commands	13
19. What This Lab Configured - and What It Did Not	14
20. Pros and Cons	15
21. PostgreSQL Official-Documentation Notes	15
22. Operational Checklist	16
23. Optional Rollback / Lab Reset	16
24. References	16

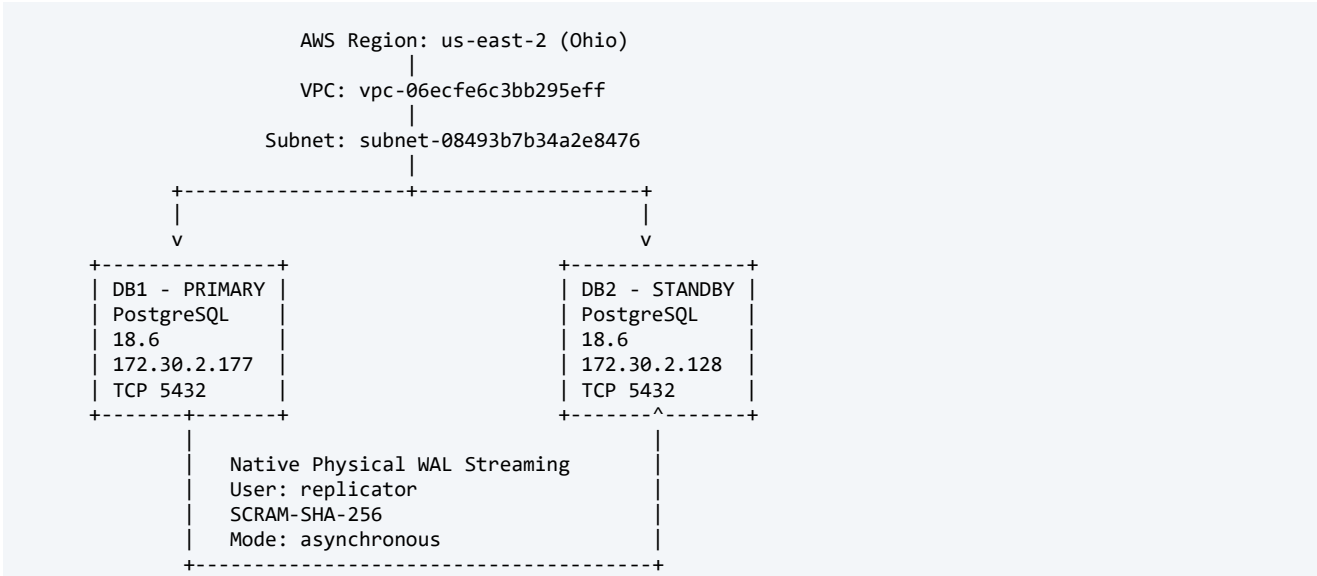
1. Purpose and Scope

This SOP documents the completed Lab-1 build of a two-node PostgreSQL 18.6 environment on Amazon EC2. DB1 is the writable primary and DB2 is a read-only hot standby receiving WAL through native PostgreSQL physical streaming replication.

Lab outcome: DB1 (172.30.2.177) streams WAL asynchronously to DB2 (172.30.2.128). DB1 reports state=streaming and sync_state=async, while DB2 reports pg_is_in_recovery()=true and accepts read-only queries.

The procedure below is written as a clean rebuild procedure. Historical command mistakes and transient connection failures from the test session are intentionally excluded.

2. Final Lab Architecture



The two EC2 instances are in the same VPC and subnet and use the same EC2 security group for database-to-database traffic. PostgreSQL listens on TCP 5432, and host-based authentication permits only the peer private address for database access.

3. Environment Inventory

Property	DB1	DB2
EC2 Name	pgsqldb1	pgsqldb2
Role	Primary	Physical Hot Standby
Hostname	db1	db2
Instance ID	i-05f19cb92d48920ba	i-00d2d4cca2f8392ee
Instance Type	t3.micro	t3.micro
Operating System	Amazon Linux 2023.12.20260831	Amazon Linux 2023.12.20260831

Property	DB1	DB2
PostgreSQL	18.6	18.6
Private IPv4	172.30.2.177	172.30.2.128
Public IPv4 used in lab	3.15.171.222	18.218.173.106
Public DNS	ec2-3-15-171-222.us-east-2.compute.amazonaws.com	ec2-18-218-173-106.us-east-2.compute.amazonaws.com
VPC	vpc-06ecfe6c3bb295eff	vpc-06ecfe6c3bb295eff
Subnet	subnet-08493b7b34a2e8476	subnet-08493b7b34a2e8476
Security Group	launch-wizard-1 / sg-074f004964aed8446	launch-wizard-1 / sg-074f004964aed8446
PostgreSQL Port	5432	5432
PGDATA	/var/lib/pgsql/data	/var/lib/pgsql/data

Lab credentials:

Purpose	User	Password	Database
Application / connectivity test	labuser	PgLab@2026	labdb
Physical replication	replicator	ReplLab@2026	Replication protocol

4. Prerequisites

- AWS account with permission to launch EC2 instances and manage security groups.
- Region selected as us-east-2 (Ohio) for this lab.
- Two EC2 instances in the same VPC and subnet.
- Key pair named pgsqlabssep2026; Windows client private key path used in this lab: C:\Users\DELL\Downloads\pgsqlabssep2026.pem.
- Internet access from the EC2 instances to the Amazon Linux repositories for package installation.
- PostgreSQL 18.6 packages available from the native Amazon Linux 2023 repository.

5. Create the AWS EC2 Lab Servers

5.1 Create or select the key pair

1. Open the Amazon EC2 console in us-east-2.
2. Go to Network & Security > Key Pairs.
3. Create or select the key pair pgsqlabssep2026.
4. For this Windows-based lab, keep the downloaded PEM file in C:\Users\DELL\Downloads.
5. Use the same key pair for DB1 and DB2 so the same SSH workflow can be used.

5.2 Launch DB1

6. Choose Launch instance.
7. Name the instance pgsqlldb1.
8. Select an Amazon Linux 2023 AMI.
9. Select t3.micro for this lab.
10. Choose the existing VPC vpc-06ecfe6c3bb295eff.
11. Choose subnet subnet-08493b7b34a2e8476.
12. Enable a public IPv4 address for direct SSH access in this training lab.
13. Choose the pgsqllabssep2026 key pair.
14. Attach the common security group described in Section 6.
15. Launch the instance and record its private IP, public IP, public DNS, and instance ID.

5.3 Launch DB2

16. Repeat the same steps and name the instance pgsqlldb2.
17. Use the same Amazon Linux 2023 AMI family, t3.micro size, VPC, subnet, key pair, and common security group.
18. Record the DB2 addressing information before proceeding.

AWS behavior: A public IP is used only for administrator SSH access in this lab. PostgreSQL replication itself uses the private VPC addresses.

6. Configure Security Group and Inbound Rules

Both instances use the same security group: launch-wizard-1 (sg-074f004964aed8446). The important design choice is that PostgreSQL and ICMP are allowed from the same security group, not from the public internet.

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	0.0.0.0/0 (lab configuration)	Administrator SSH from the local workstation
PostgreSQL	TCP	5432	sg-074f004964aed8446	DB1 ↔ DB2 PostgreSQL traffic over private VPC networking
All ICMP - IPv4	ICMP	All	sg-074f004964aed8446	Private hostname/IP reachability testing between DB1 and DB2

Outbound rule used in the lab:

Type	Protocol	Port	Destination
All traffic	All	All	0.0.0.0/0

Security-group rule: Do not expose PostgreSQL TCP/5432 to 0.0.0.0/0. The lab uses the security-group ID itself as the source so only network interfaces associated with that security group can initiate the permitted traffic.

6.1 Console procedure

19. EC2 Console > Security Groups > select launch-wizard-1.
20. Open Inbound rules > Edit inbound rules.
21. Keep/add SSH TCP 22 for the lab administrator path.

22. Add PostgreSQL TCP 5432; set Source to Custom and select launch-wizard-1 / sg-074f004964aed8446.
23. Add All ICMP - IPv4; set Source to the same security group.
24. Save rules.
25. Attach this same security group to both DB1 and DB2.

7. Connect to DB1 and DB2 by SSH

From Windows PowerShell:

```
cd C:\Users\DELL\Downloads
ssh -i "pgsqlabssep2026.pem" ec2-user@ec2-3-15-171-222.us-east-2.compute.amazonaws.com
ssh -i "pgsqlabssep2026.pem" ec2-user@ec2-18-218-173-106.us-east-2.compute.amazonaws.com
```

After login, verify the OS:

```
cat /etc/os-release
uname -r
```

8. Configure Hostnames and Private Name Resolution

8.1 DB1

```
sudo hostnamectl set-hostname db1
hostname
cat /etc/hostname
```

8.2 DB2

```
sudo hostnamectl set-hostname db2
hostname
cat /etc/hostname
```

8.3 Configure /etc/hosts on both servers

```
sudo vi /etc/hosts

# Add:
172.30.2.177    db1
172.30.2.128    db2
```

8.4 Validate private connectivity

On DB1:

```
getent hosts db2
ping -c 4 db2
```

On DB2:

```
getent hosts db1
ping -c 4 db1
```

Expected result: four successful replies and 0% packet loss after the ICMP security-group rule is in place.

9. Install PostgreSQL 18.6 on Both Servers

Run the following on both DB1 and DB2.

```
sudo dnf clean all
sudo dnf makecache

dnf search postgresql18
dnf list available 'postgresql18*'
```

Install the PostgreSQL server, client, and contrib packages:

```
sudo dnf install -y postgresql18 postgresql18-server postgresql18-contrib
```

Validate the installation:

```
rpm -qa | grep postgresql18 | sort
psql --version
postgres --version
```

Expected PostgreSQL version for this lab:

```
psql (PostgreSQL) 18.6
postgres (PostgreSQL) 18.6
```

Package source: In this completed lab, PostgreSQL 18.6 was supplied by the native Amazon Linux 2023 repository; no external PGDG repository was required.

10. Initialize and Start PostgreSQL

Run on DB1 and DB2 before converting DB2 to a standby:

```
sudo postgresql-setup --initdb
sudo systemctl enable postgresql
sudo systemctl start postgresql
sudo systemctl status postgresql --no-pager
```

Validate server version and configuration paths:

```
sudo -u postgres psql -c "SELECT version();"
sudo -u postgres psql -Atc "SHOW data_directory;"
sudo -u postgres psql -Atc "SHOW config_file;"
sudo -u postgres psql -Atc "SHOW hba_file;"
sudo ss -lntp | grep 5432
```

Expected paths:

```
/var/lib/pgsql/data
/var/lib/pgsql/data/postgresql.conf
/var/lib/pgsql/data/pg_hba.conf
```

11. Configure PostgreSQL Network Listening

PostgreSQL initially listened only on localhost. Configure both nodes to accept TCP connections on their network interfaces.

On DB1 and DB2:

```
sudo vi /var/lib/pgsql/data/postgresql.conf

# Set:
listen_addresses = '*'
```

Restart PostgreSQL because listen_addresses is a server-start parameter:

```
sudo systemctl restart postgresql
sudo -u postgres psql -c "SHOW listen_addresses;"
sudo ss -lntp | grep 5432
```

Expected:

```
listen_addresses
-----
*

LISTEN ... 0.0.0.0:5432 ...
LISTEN ... [::]:5432 ...
```

PostgreSQL behavior: listen_addresses='*' means PostgreSQL listens on all available IP interfaces. Access is still controlled by pg_hba.conf and the EC2 security group.

12. Configure pg_hba.conf and Validate Node Connectivity

12.1 DB1 - allow DB2

```
sudo vi /var/lib/pgsql/data/pg_hba.conf

# DB2 access to DB1
host    all    all    172.30.2.128/32    scram-sha-256
```

12.2 DB2 - allow DB1

```
sudo vi /var/lib/pgsql/data/pg_hba.conf

# DB1 access to DB2
host    all    all    172.30.2.177/32    scram-sha-256
```

12.3 Validate HBA syntax

```
sudo -u postgres psql -c "
SELECT line_number, type, database, user_name, address, auth_method, error
FROM pg_hba_file_rules
WHERE error IS NOT NULL;
"
```

Expected: (0 rows). Then reload:

```
sudo systemctl reload postgresql
```

12.4 Raw TCP tests

DB1 → DB2:

```
timeout 3 bash -c '</dev/tcp/db2/5432' && echo "DB1 -> DB2 PostgreSQL port OPEN" || echo "DB1 -> DB2 PostgreSQL port FAILED"
```

DB2 → DB1:

```
timeout 3 bash -c '</dev/tcp/db1/5432' && echo "DB2 -> DB1 PostgreSQL port OPEN" || echo "DB2 -> DB1 PostgreSQL port FAILED"
```

13. Create the Lab Database and Test Login

Create the lab role and database on DB1. They will later be included automatically in the physical base backup copied to DB2.

```
sudo -u postgres psql

CREATE ROLE labuser
WITH LOGIN
PASSWORD 'PgLab@2026';

\du labuser
\q

sudo -u postgres createdb -O labuser labdb
sudo -u postgres psql -c "\l labdb"
```

From DB2, test an authenticated connection to DB1:

```
PGPASSWORD='PgLab@2026' psql -h db1 -p 5432 -U labuser -d labdb -c "SELECT inet_server_addr(),
inet_server_port(), current_database(), current_user;"
```

Expected server address: 172.30.2.177.

14. Prepare DB1 as the Replication Primary

14.1 Validate replication-related parameters

```
sudo -u postgres psql -c "
SELECT
    name AS parameter,
    setting AS current_value
FROM pg_settings
WHERE name IN
(
    'wal_level',
    'max_wal_senders',
    'max_replication_slots',
    'hot_standby',
    'wal_keep_size'
)
ORDER BY name;
"
```

Validated values in this lab:

Parameter	Value
hot_standby	on
max_replication_slots	10
max_wal_senders	10
wal_keep_size	0
wal_level	replica

14.2 Create the replication role

```
sudo -u postgres psql

CREATE ROLE replicator
WITH
    REPLICATION
    LOGIN
    PASSWORD 'ReplLab@2026';

\du replicator
\q
```

14.3 Add the physical replication HBA rule

```
sudo vi /var/lib/pgsql/data/pg_hba.conf

# DB2 Physical Streaming Replication
host    replication    replicator    172.30.2.128/32    scram-sha-256
```

Validate and reload:

```
sudo -u postgres psql -c "
SELECT line_number, type, database, user_name, address, auth_method, error
FROM pg_hba_file_rules
WHERE error IS NOT NULL;
"

sudo systemctl reload postgresql

sudo -u postgres psql -c "
SELECT pg_is_in_recovery() AS is_in_recovery;
"
```

Expected on DB1: `is_in_recovery = f`.

14.4 Keep backup files outside PGDATA

Before taking a physical base backup, ensure PostgreSQL can read everything under PGDATA. For this lab, configuration backup copies were moved outside PGDATA.

```
sudo mkdir -p /var/lib/pgsql/config_backups
sudo chown postgres:postgres /var/lib/pgsql/config_backups
sudo chmod 700 /var/lib/pgsql/config_backups

sudo find /var/lib/pgsql/data -xdev \( -user root -o -group root \) -ls
```

Expected: no unexpected root-owned files inside `/var/lib/pgsql/data`.

15. Convert DB2 into the Physical Standby

Important: This step replaces DB2's independent PostgreSQL cluster with a physical copy of DB1. The lab preserves the original DB2 data directory first.

15.1 Stop DB2

```
sudo systemctl stop postgresql
sudo systemctl is-active postgresql
sudo ss -lntp | grep 5432 || echo "Port 5432 is not listening"
```

Expected: inactive and no listener on 5432.

15.2 Preserve the original DB2 cluster

```
sudo mv /var/lib/pgsql/data /var/lib/pgsql/data_before_replication_20260902

sudo test -d /var/lib/pgsql/data_before_replication_20260902 && echo "BACKUP DIRECTORY EXISTS" || echo "BACKUP DIRECTORY NOT FOUND"
```

15.3 Create a fresh empty PGDATA

```
sudo mkdir /var/lib/pgsql/data
sudo chown postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data

sudo find /var/lib/pgsql/data -mindepth 1 -maxdepth 1 -print
```

Expected: no output from find.

15.4 Create the replication password file

```
sudo -u postgres bash -c 'echo "db1:5432:replication:replicator:ReplLab@2026" > /var/lib/pgsql/.pgpass'

sudo chown postgres:postgres /var/lib/pgsql/.pgpass
sudo chmod 600 /var/lib/pgsql/.pgpass
sudo ls -l /var/lib/pgsql/.pgpass
```

15.5 Validate the physical replication protocol

```
sudo -u postgres psql "host=db1 port=5432 user=replicator replication=true" -c "IDENTIFY_SYSTEM;"
```

Expected: one row containing DB1's system identifier, timeline, and current WAL location.

15.6 Take the base backup from DB1

```
sudo -u postgres pg_basebackup -h db1 -p 5432 -U replicator -D /var/lib/pgsql/data -Fp -Xs -P -R
```

The `-R` option prepares the backup as a standby by writing the required standby configuration, including `standby.signal` and primary connection information.

15.7 Validate the copied standby

```
sudo test -f /var/lib/pgsql/data/standby.signal && echo "standby.signal EXISTS" || echo "standby.signal MISSING"

sudo cat /var/lib/pgsql/data/PG_VERSION
sudo ls -ld /var/lib/pgsql/data

sudo grep "primary_conninfo" /var/lib/pgsql/data/postgresql.auto.conf
```

Expected: standby.signal exists, PG_VERSION is 18, and PGDATA is owned by postgres:postgres.

16. Start and Validate Streaming Replication

16.1 Start DB2

```
sudo systemctl start postgresql
sudo systemctl status postgresql --no-pager
```

16.2 Validate DB2 recovery state

```
sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    pg_last_wal_receive_lsn() AS receive_lsn,
    pg_last_wal_replay_lsn() AS replay_lsn;
"
```

Validated lab result:

is_standby	receive_lsn	replay_lsn
t	0/4000060	0/4000060

16.3 Validate DB1 sender state

```
sudo -u postgres psql -c "
SELECT
    username,
    application_name,
    client_addr,
    state,
    sync_state
FROM pg_stat_replication;
"
```

Validated lab result:

username	application_name	client_addr	state	sync_state
replicator	walreceiver	172.30.2.128	streaming	async

17. End-to-End Data Replication Validation

17.1 Create and populate a validation table on DB1

```
sudo -u postgres psql -d labdb

CREATE TABLE IF NOT EXISTS replication_validation
(
    id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    message     text NOT NULL,
    created_at  timestampz DEFAULT current_timestamp
);

INSERT INTO replication_validation(message)
VALUES
('Inserted on DB1 Primary'),
```

```
('PostgreSQL 18.6 Streaming Replication Test');

SELECT * FROM replication_validation;
\q
```

17.2 Read the same data on DB2

```
sudo -u postgres psql -d labdb -c "
SELECT id, message, created_at
FROM replication_validation
ORDER BY id;
"
```

Validated result: both rows written on DB1 were visible on DB2.

17.3 Confirm hot-standby read-only behavior

```
sudo -u postgres psql -d labdb -c "
SHOW transaction_read_only;
"
```

Expected: on.

A write attempted on DB2 should be rejected because the server is in hot-standby recovery mode.

18. Replication Health-Check Commands

18.1 Primary-side health query (DB1)

```
sudo -u postgres psql -c "
SELECT
    client_addr AS standby_ip,
    state,
    sync_state AS mode,
    pg_size_pretty(
        pg_wal_lsn_diff(sent_lsn, replay_lsn)
    ) AS replication_lag
FROM pg_stat_replication;
"
```

Validated lab output:

standby_ip	state	mode	replication_lag
172.30.2.128	streaming	async	0 bytes

18.2 Standby-side receiver query (DB2)

```
sudo -u postgres psql -c "
SELECT
    status,
    sender_host,
    sender_port,
    latest_end_lsn,
    slot_name
FROM pg_stat_wal_receiver;
"
```

18.3 Standby replay status (DB2)

```
sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    pg_last_wal_receive_lsn() AS receive_lsn,
    pg_last_wal_replay_lsn() AS replay_lsn,
    pg_size_pretty(
        pg_wal_lsn_diff(
            pg_last_wal_receive_lsn(),
            pg_last_wal_replay_lsn()
        )
    ) AS replay_lag
FROM pg_stat_wal_receiver;
"
```

```

) AS replay_lag;
"

```

Correct view usage: `pg_stat_replication` is a primary-side view with one row per WAL sender. `pg_stat_wal_receiver` is the standby-side view and normally contains one row for the active WAL receiver.

19. What This Lab Configured - and What It Did Not

Capability	Status	Details
PostgreSQL physical streaming replication	Configured	DB1 primary → DB2 standby
Hot standby / read-only queries	Configured	DB2 accepts read-only SQL while replaying WAL
SCRAM-SHA-256 authentication	Configured	Peer database and replication connections
Asynchronous replication	Configured	<code>sync_state=async</code>
Automatic failover	Not configured	PostgreSQL core does not provide automatic failure detection/orchestration
Virtual IP / load balancer	Not configured	Clients are not automatically redirected after failover
Replication slot	Not configured	<code>max_replication_slots</code> is available, but no physical slot was created
WAL archive / <code>archive_command</code>	Not configured in Lab-1	Streaming replication is the active transport
Patroni / etcd / Consul	Not configured	Outside Lab-1 scope
HAProxy / PgBouncer / Keepalived	Not configured	Outside Lab-1 scope

20. Pros and Cons

20.1 Pros

- Native PostgreSQL feature; no external replication product is required for the database-level streaming mechanism.
- Physical replication copies the whole PostgreSQL cluster at the WAL/block level, so databases and objects remain consistent with the primary.
- Hot standby enables read-only reporting or validation queries on DB2.
- Asynchronous mode generally has lower commit latency than synchronous replication because primary commits do not wait for standby acknowledgement.
- The standby can be promoted manually if the primary becomes unavailable.
- Monitoring is available through built-in views such as `pg_stat_replication` and `pg_stat_wal_receiver`.

20.2 Cons / Limitations

- Asynchronous replication can lose the most recent transactions during an abrupt primary failure if their WAL has not reached/replayed on the standby.
- Physical streaming replication operates at the PostgreSQL cluster level; it is not a per-table replication mechanism.
- DB2 is read-only while in hot standby and cannot accept normal application writes.
- Automatic failover, fencing, client redirection, and primary election are not provided by PostgreSQL core.
- Without a permanent replication slot or sufficient WAL retention/archive, a long-disconnected standby can fall behind beyond available WAL and may require rebuilding.
- Long-running queries on a hot standby can conflict with WAL replay and can be canceled depending on standby delay settings.

21. PostgreSQL Official-Documentation Notes

- PostgreSQL 18 documentation defines primary servers as senders and standby servers as receivers for built-in streaming replication.
- For physical streaming replication, the primary must allow replication authentication, normally with a dedicated role and an HBA record whose database field is replication.
- `listen_addresses` controls the network interfaces on which PostgreSQL accepts TCP connections; '*' means all available interfaces.
- `pg_hba.conf` is re-read on reload/SIGHUP, while `listen_addresses` is a server-start parameter and therefore requires a restart.
- Hot standby allows read-only queries while the server is in recovery. A standby with `standby.signal` and `hot_standby=on` becomes a hot standby after reaching a consistent recovery state.
- `pg_stat_replication` reports one row per WAL sender on the sending server; `pg_stat_wal_receiver` reports the receiving connection on the standby.
- PostgreSQL does not include automatic failure detection and failover orchestration software. Promotion can be triggered manually with `pg_ctl promote` or `pg_promote()`.
- For robust long-disconnection tolerance, PostgreSQL provides mechanisms such as WAL retention, WAL archiving, and replication slots; these are separate design decisions from the basic Lab-1 configuration.

22. Operational Checklist

Check	DB1 Primary	DB2 Standby	Expected
Service	<code>systemctl is-active postgresql</code>	<code>systemctl is-active postgresql</code>	active
Role	<code>SELECT pg_is_in_recovery();</code>	<code>SELECT pg_is_in_recovery();</code>	DB1=f, DB2=t
Port	<code>ss -lntp grep 5432</code>	<code>ss -lntp grep 5432</code>	5432 listening
Sender/receiver	<code>pg_stat_replication</code>	<code>pg_stat_wal_receiver</code>	streaming
Replication lag	<code>pg_stat_replication</code>	receive/replay LSN	Near 0 in idle lab
Read/write	Writable	Read-only	Expected
Test data	INSERT	SELECT	Rows appear on DB2

23. Optional Rollback / Lab Reset

If you want to return DB2 to its original independent cluster created before replication:

```
# DB2 only
sudo systemctl stop postgresql

sudo mv /var/lib/pgsql/data /var/lib/pgsql/data_streaming_standby
sudo mv /var/lib/pgsql/data_before_replication_20260902 /var/lib/pgsql/data

sudo chown -R postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data

sudo systemctl start postgresql
sudo systemctl status postgresql --no-pager
```

Caution: Use rollback only for lab reset. After the standby has been used for a long period, the preserved independent DB2 cluster is stale and must not be treated as a synchronized copy of DB1.

24. References

PostgreSQL official documentation:

- [PostgreSQL 18 - High Availability, Load Balancing, and Replication](#)
- [PostgreSQL 18 - Replication Configuration](#)
- [PostgreSQL 18 - Connections and Authentication / listen_addresses](#)
- [PostgreSQL 18 - pg_hba.conf](#)
- [PostgreSQL 18 - Hot Standby](#)
- [PostgreSQL 18 - Monitoring Statistics](#)
- [PostgreSQL 18 - Failover](#)
- [PostgreSQL 18 - pg_basebackup](#)

AWS official documentation used for EC2/security-group creation:

- [Launch an EC2 instance using the launch instance wizard](#)
- [Create a security group for an EC2 instance](#)
- [Amazon EC2 key pairs](#)

Lab evidence reviewed: DB1 and DB2 command outputs supplied by the user, plus the completed validation results from the Lab-1 session.

Appendix A - Complete Execution Scripts Used in Lab-1

This appendix consolidates the successful DB1 and DB2 command sequences used in the completed lab. Repeated commands and transient failed attempts are excluded so the appendix can be used as a clean rebuild script reference.

A.1 DB1 - Primary Server Commands

```
# =====
# LAB-1 - DB1 PRIMARY SERVER
# PostgreSQL 18.6 / Amazon Linux 2023
# Host: db1
# Private IP: 172.30.2.177
# =====

# 1. Set hostname
sudo hostnamectl set-hostname db1
hostname
cat /etc/hostname
hostname -I

# 2. Configure local host resolution
sudo vi /etc/hosts

# Add these entries:
172.30.2.177    db1
172.30.2.128    db2

# Validate name resolution and connectivity
getent hosts db1
getent hosts db2
ping -c 4 db2

# 3. Validate OS
cat /etc/os-release

# 4. Refresh Amazon Linux package metadata
sudo dnf clean all
sudo dnf makecache

# 5. Confirm PostgreSQL 18 packages are available
dnf search postgresql18
dnf list available 'postgresql18*'

# 6. Install PostgreSQL 18.6
sudo dnf install -y postgresql18 postgresql18-server postgresql18-contrib

# 7. Validate installed PostgreSQL packages/version
rpm -qa | grep postgresql18 | sort
psql --version
postgres --version

# 8. Initialize PostgreSQL cluster
sudo postgresql-setup --initdb

# 9. Enable and start PostgreSQL
sudo systemctl enable postgresql
sudo systemctl start postgresql
sudo systemctl status postgresql --no-pager

# 10. Validate PostgreSQL
sudo -u postgres psql -c "SELECT version();"

# 11. Validate important paths
sudo -u postgres psql -Atc "SHOW data_directory;"
sudo -u postgres psql -Atc "SHOW config_file;"
sudo -u postgres psql -Atc "SHOW hba_file;"

# Expected:
# /var/lib/pgsql/data
# /var/lib/pgsql/data/postgresql.conf
# /var/lib/pgsql/data/pg_hba.conf
```



```
# 12. Configure PostgreSQL to listen on network interfaces
sudo vi /var/lib/pgsql/data/postgresql.conf
```

```
# Set:
listen_addresses = '*'
```

```
# Restart because listen_addresses requires restart
sudo systemctl restart postgresql
```

```
# Validate
sudo -u postgres psql -c "SHOW listen_addresses;"
sudo -u postgres psql -c "SHOW port;"
sudo ss -lntp | grep 5432
```

```
# 13. Allow DB2 database connectivity in pg_hba.conf
sudo vi /var/lib/pgsql/data/pg_hba.conf
```

```
# Add:
host    all    all    172.30.2.128/32    scram-sha-256
```

```
# Validate HBA syntax
sudo -u postgres psql -c "
SELECT
    line_number,
    type,
    database,
    user_name,
    address,
    auth_method,
    error
FROM pg_hba_file_rules
WHERE error IS NOT NULL;
"
```

```
# Reload HBA
sudo systemctl reload postgresql
```

```
# Validate DB2 rule
sudo -u postgres psql -c "
SELECT
    line_number,
    type,
    database,
    user_name,
    address,
    auth_method
FROM pg_hba_file_rules
WHERE address = '172.30.2.128';
"
```

```
# 14. Validate DB1 -> DB2 TCP/5432
timeout 3 bash -c '</dev/tcp/db2/5432' \
&& echo "DB1 -> DB2 PostgreSQL port OPEN" \
|| echo "DB1 -> DB2 PostgreSQL port FAILED"
```

```
# 15. Create lab login
sudo -u postgres psql
```

```
CREATE ROLE labuser
WITH
LOGIN
PASSWORD 'PgLab@2026';
```

```
\du labuser
\q
```

```
# 16. Create lab database
sudo -u postgres createdb -O labuser labdb
sudo -u postgres psql -c "\l labdb"
```

```
# 17. Check replication prerequisite parameters
sudo -u postgres psql -c "
SELECT
    name AS parameter,
```

```

        setting AS current_value
FROM pg_settings
WHERE name IN
(
    'wal_level',
    'max_wal_senders',
    'max_replication_slots',
    'hot_standby',
    'wal_keep_size'
)
ORDER BY name;
"

# Lab values validated:
# hot_standby          = on
# max_replication_slots = 10
# max_wal_senders      = 10
# wal_keep_size        = 0
# wal_level            = replica

# 18. Create physical replication account
sudo -u postgres psql

CREATE ROLE replicator
WITH
    REPLICATION
    LOGIN
    PASSWORD 'ReplLab@2026';

\du replicator
\q

# 19. Authorize DB2 for physical streaming replication
sudo vi /var/lib/pgsql/data/pg_hba.conf

# Add:
host      replication      replicator      172.30.2.128/32      scram-sha-256

# Validate HBA syntax
sudo -u postgres psql -c "
SELECT
    line_number,
    type,
    database,
    user_name,
    address,
    auth_method,
    error
FROM pg_hba_file_rules
WHERE error IS NOT NULL;
"

# Validate replication rule
sudo -u postgres psql -c "
SELECT
    line_number,
    type,
    database,
    user_name,
    address,
    auth_method
FROM pg_hba_file_rules
WHERE 'replication' = ANY(database);
"

sudo systemctl reload postgresql

# 20. Confirm DB1 is PRIMARY
sudo -u postgres psql -c "
SELECT pg_is_in_recovery() AS is_in_recovery;
"

# Expected: f

```

```

# 21. Store manual configuration backups outside PGDATA
sudo mkdir -p /var/lib/pgsql/config_backups
sudo chown postgres:postgres /var/lib/pgsql/config_backups
sudo chmod 700 /var/lib/pgsql/config_backups

# Check PGDATA for unexpected root-owned objects before pg_basebackup
sudo find /var/lib/pgsql/data \
  -xdev \
  \( -user root -o -group root \) \
  -ls

# Expected: no unexpected root-owned files

# 22. After DB2 starts as standby, check streaming status
sudo -u postgres psql -c "
SELECT
    username,
    application_name,
    client_addr,
    state,
    sync_state
FROM pg_stat_replication;
"

# Expected:
# replicator | walreceiver | 172.30.2.128 | streaming | async

# 23. Create end-to-end validation table and test rows
sudo -u postgres psql -d labdb

CREATE TABLE IF NOT EXISTS replication_validation
(
    id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    message     text NOT NULL,
    created_at  timestampz DEFAULT current_timestamp
);

INSERT INTO replication_validation(message)
VALUES
('Inserted on DB1 Primary'),
('PostgreSQL 18.6 Streaming Replication Test');

SELECT * FROM replication_validation;
\q

# 24. Detailed primary replication health check
sudo -u postgres psql -c "
SELECT
    application_name,
    client_addr,
    state,
    sync_state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(
        pg_wal_lsn_diff(sent_lsn, replay_lsn)
    ) AS replay_lag_bytes
FROM pg_stat_replication;
"

# 25. Compact primary replication health check
sudo -u postgres psql -c "
SELECT
    client_addr AS standby_ip,
    state,
    sync_state AS mode,
    pg_size_pretty(
        pg_wal_lsn_diff(sent_lsn, replay_lsn)
    ) AS replication_lag
FROM pg_stat_replication;
"

```

```
# Validated Lab-1 output:
# 172.30.2.128 | streaming | async | 0 bytes
```

A.2 DB2 - Physical Hot Standby Commands

```
# =====
# LAB-1 - DB2 PHYSICAL HOT STANDBY
# PostgreSQL 18.6 / Amazon Linux 2023
# Host: db2
# Private IP: 172.30.2.128
# =====

# 1. Set hostname
sudo hostnamectl set-hostname db2
hostname
cat /etc/hostname
hostname -I

# 2. Configure local host resolution
sudo vi /etc/hosts

# Add:
172.30.2.177    db1
172.30.2.128    db2

# Validate
getent hosts db1
getent hosts db2
ping -c 4 db1

# 3. Validate OS
cat /etc/os-release

# 4. Refresh repository metadata
sudo dnf clean all
sudo dnf makecache

# 5. Confirm PostgreSQL 18 packages
dnf search postgresql18
dnf list available 'postgresql18*'

# 6. Install PostgreSQL 18.6
sudo dnf install -y postgresql18 postgresql18-server postgresql18-contrib

# 7. Validate packages/version
rpm -qa | grep postgresql18 | sort
psql --version
postgres --version

# 8. Initialize the initial DB2 cluster
sudo postgresql-setup --initdb

# 9. Enable/start PostgreSQL
sudo systemctl enable postgresql
sudo systemctl start postgresql
sudo systemctl status postgresql --no-pager

# 10. Validate PostgreSQL and paths
sudo -u postgres psql -c "SELECT version();"
sudo -u postgres psql -Atc "SHOW data_directory;"
sudo -u postgres psql -Atc "SHOW config_file;"
sudo -u postgres psql -Atc "SHOW hba_file;"
sudo ss -lntp | grep 5432

# 11. Configure network listening for initial connectivity test
sudo vi /var/lib/pgsql/data/postgresql.conf

# Set:
listen_addresses = '*'

sudo systemctl restart postgresql
sudo -u postgres psql -c "SHOW listen_addresses;"
sudo ss -lntp | grep 5432
```

```
# 12. Allow DB1 database connectivity during pre-replication testing
sudo vi /var/lib/pgsql/data/pg_hba.conf

# Add:
host    all    all    172.30.2.177/32    scram-sha-256

# Validate
sudo -u postgres psql -c "
SELECT
    line_number,
    type,
    database,
    user_name,
    address,
    auth_method,
    error
FROM pg_hba_file_rules
WHERE error IS NOT NULL;
"

sudo systemctl reload postgresql

# 13. Validate DB2 -> DB1 TCP/5432
timeout 3 bash -c '</dev/tcp/db1/5432' \
&& echo "DB2 -> DB1 PostgreSQL port OPEN" \
|| echo "DB2 -> DB1 PostgreSQL port FAILED"

# 14. Pre-replication authenticated DB1 connection test
PGPASSWORD='PgLab@2026' \
psql -h db1 -p 5432 -U labuser -d labdb \
-c "SELECT inet_server_addr(), current_database(), current_user;"

# 15. Stop DB2 before replacing its independent cluster
sudo systemctl stop postgresql
sudo systemctl is-active postgresql
sudo ss -lntp | grep 5432 || echo "Port 5432 is not listening"

# 16. Preserve original independent DB2 PGDATA
sudo mv /var/lib/pgsql/data \
/var/lib/pgsql/data_before_replication_20260902

# Validate preserved directory
sudo test -d /var/lib/pgsql/data_before_replication_20260902 \
&& echo "BACKUP DIRECTORY EXISTS" \
|| echo "BACKUP DIRECTORY NOT FOUND"

sudo ls -la /var/lib/pgsql/data_before_replication_20260902 | head -20

# 17. Create a new empty PGDATA
sudo mkdir /var/lib/pgsql/data
sudo chown postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data

# Validate empty target
sudo find /var/lib/pgsql/data -mindepth 1 -maxdepth 1 -print

# 18. Create .pgpass for replication
sudo -u postgres bash -c \
'echo "db1:5432:replication:replicator:ReplLab@2026" > /var/lib/pgsql/.pgpass'

sudo chown postgres:postgres /var/lib/pgsql/.pgpass
sudo chmod 600 /var/lib/pgsql/.pgpass
sudo ls -l /var/lib/pgsql/.pgpass

# 19. Validate the PostgreSQL physical replication protocol
sudo -u postgres psql \
"host=db1 port=5432 user=replicator replication=true" \
-c "IDENTIFY_SYSTEM;"

# Expected: DB1 systemid/timeline/current WAL position

# 20. Take physical base backup from DB1 and create standby config
sudo -u postgres pg_basebackup \
-h db1 \
```

```

-p 5432 \
-U replicator \
-D /var/lib/pgsql/data \
-Fp \
-Xs \
-P \
-R

# 21. Validate standby preparation
sudo test -f /var/lib/pgsql/data/standby.signal \
&& echo "standby.signal EXISTS" \
|| echo "standby.signal MISSING"

sudo cat /var/lib/pgsql/data/PG_VERSION
sudo ls -ld /var/lib/pgsql/data

# Validate generated primary_conninfo
sudo grep "primary_conninfo" \
/var/lib/pgsql/data/postgresql.auto.conf

# 22. Start DB2 as the physical standby
sudo systemctl start postgresql
sudo systemctl status postgresql --no-pager

# 23. Validate standby/recovery state and LSNs
sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    pg_last_wal_receive_lsn() AS receive_lsn,
    pg_last_wal_replay_lsn() AS replay_lsn;
"

# Expected:
# is_standby = t

# 24. Validate replicated rows
sudo -u postgres psql -d labdb -c "
SELECT
    id,
    message,
    created_at
FROM replication_validation
ORDER BY id;
"

# 25. Confirm DB2 is read-only
sudo -u postgres psql -d labdb -c "
SHOW transaction_read_only;
"

# Expected:
# on

# 26. Optional negative validation - write must fail on standby
sudo -u postgres psql -d labdb -c "
INSERT INTO replication_validation(message)
VALUES ('Attempted write on DB2');
"

# Expected:
# ERROR: cannot execute INSERT in a read-only transaction

# 27. NOTE: pg_stat_replication is a PRIMARY-side view.
# Running it on DB2 correctly returns 0 rows.
sudo -u postgres psql -c "
SELECT
    client_addr AS standby_ip,
    state,
    sync_state AS mode,
    pg_size_pretty(
        pg_wal_lsn_diff(sent_lsn, replay_lsn)
    ) AS replication_lag
FROM pg_stat_replication;
"

```

```
# Expected on DB2:
# (0 rows)

# 28. Correct standby-side receiver view
sudo -u postgres psql -c "
SELECT
    status,
    sender_host,
    sender_port,
    latest_end_lsn,
    slot_name
FROM pg_stat_wal_receiver;
"

# 29. Standby receive/replay lag check
sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    pg_last_wal_receive_lsn() AS receive_lsn,
    pg_last_wal_replay_lsn() AS replay_lsn,
    pg_size_pretty(
        pg_wal_lsn_diff(
            pg_last_wal_receive_lsn(),
            pg_last_wal_replay_lsn()
        )
    ) AS replay_lag;
"
```

A.3 Windows PowerShell SSH Commands

```
cd C:\Users\DELL\Downloads

# DB1
ssh -i "pgsqlabssep2026.pem" \
ec2-user@ec2-3-15-171-222.us-east-2.compute.amazonaws.com

# DB2
ssh -i "pgsqlabssep2026.pem" \
ec2-user@ec2-18-218-173-106.us-east-2.compute.amazonaws.com
```

Appendix B - Screenshot Evidence from the Completed Lab

The following screenshots are the AWS console and terminal evidence available from the completed Lab-1 session. They are retained as implementation evidence next to the corresponding configuration areas.

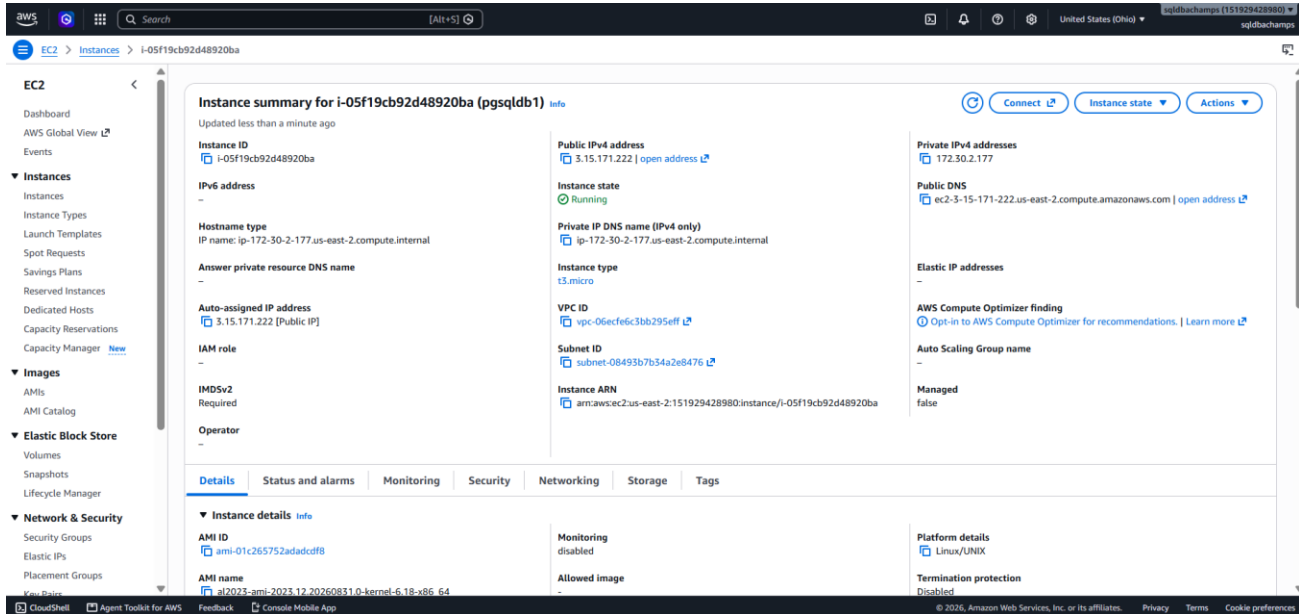


Figure B-1 - AWS EC2 DB1 instance summary showing the DB1 server and network details.

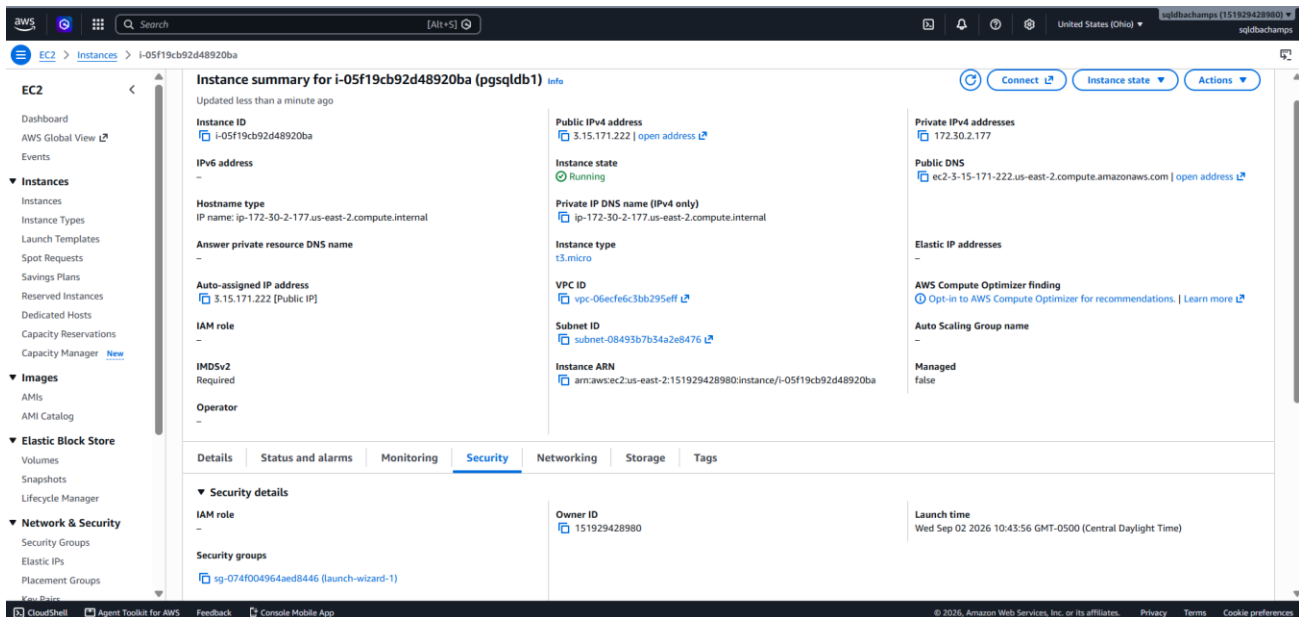


Figure B-2 - AWS EC2 DB2 instance summary showing the DB2 server and network details.

Connect to Linux instance [Info](#)

Connect to an instance to test, troubleshoot, deploy code, update software etc. You can use browser based methods or an SSH client.

Instance info

Instance: [i-00d2d4cca2f8392ee](#) (pgsqlb2)
 State: Running
 Status: 3/3 checks passed

VPC: [vpc-06ecfec3bb295eff](#)

Subnet: [subnet-08493b7b34a2e8476](#)

Security groups: [sg-08cbde6ff7ab9cbcd](#) (default)

SSH info

SSH key pair assigned at launch: [pgsqlabssep2026](#)

Public DNS: [ec2-18-218-173-106.us-east-2.compute.amazonaws.com](#)

Username: [ec2-user](#)

[Connect assist](#)

How to connect

1. Open an SSH client
2. Locate or download your private key file
3. Ensure your key is not publicly viewable
4. Connect to your instance using its Public DNS

```
chmod 400 "pgsqlabssep2026.pem"
```

```
ssh -i "pgsqlabssep2026.pem" ec2-user@ec2-18-218-173-106.us-east-2.compute.amazonaws.com
```

[Copied command](#)

Figure B-3 - AWS EC2 Connect-to-instance page used to confirm the Linux SSH connection method.

EC2

Dashboard
 AWS Global View [L](#)
 Events

Instances

Instances
 Instance Types
 Launch Templates
 Spot Requests
 Savings Plans
 Reserved Instances
 Dedicated Hosts
 Capacity Reservations
 Capacity Manager [New](#)

Images

AMIs
 AMI Catalog

Elastic Block Store

Volumes
 Snapshots
 Lifecycle Manager

Network & Security

Security Groups
 Elastic IPs
 Placement Groups

Operator

Details | Status and alarms | Monitoring | **Security** | Networking | Storage | Tags

Security details

IAM role: [-](#)

Owner ID: [151929428980](#)

Launch time: Wed Sep 02 2026 10:43:56 GMT-0500 (Central Daylight Time)

Security groups: [sg-074f004964aed8446](#) (launch-wizard-1)

Inbound rules

[Filter rules](#)

Name	Security group rule ID	Port range	Protocol	Source	Security groups	Description
-	sgr-05d54c4489a211ae6	22	TCP	0.0.0.0/0	launch-wizard-1	-

Outbound rules

[Filter rules](#)

Name	Security group rule ID	Port range	Protocol	Destination	Security groups	Description
-	sgr-077e6ecd7cee3ff26	All	All	0.0.0.0/0	launch-wizard-1	-

Figure B-4 - EC2 security configuration associated with the lab instance.

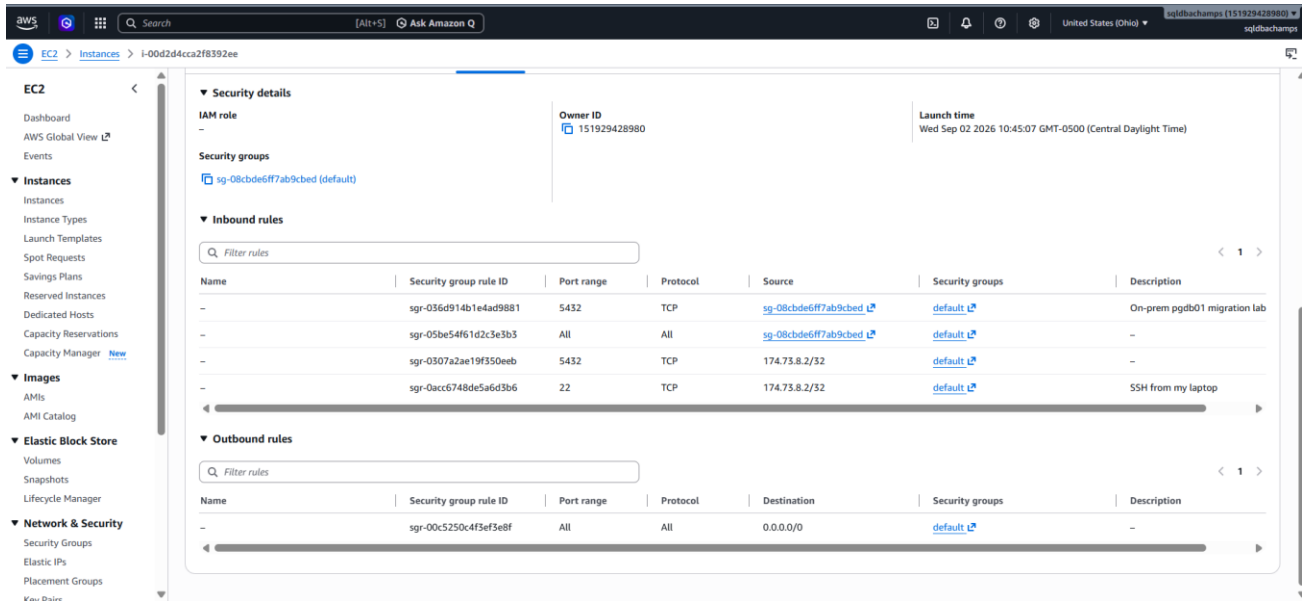


Figure B-5 - AWS EC2 Security tab showing the security group and inbound/outbound rule context.

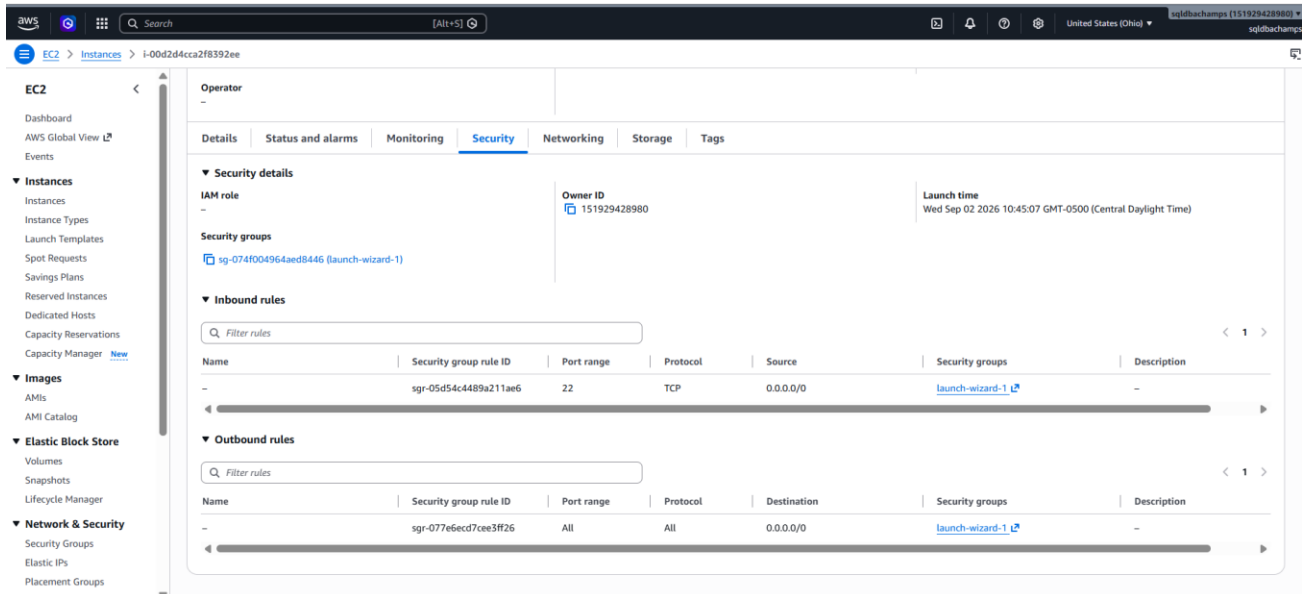


Figure B-6 - AWS EC2 Security tab evidence for the peer instance.

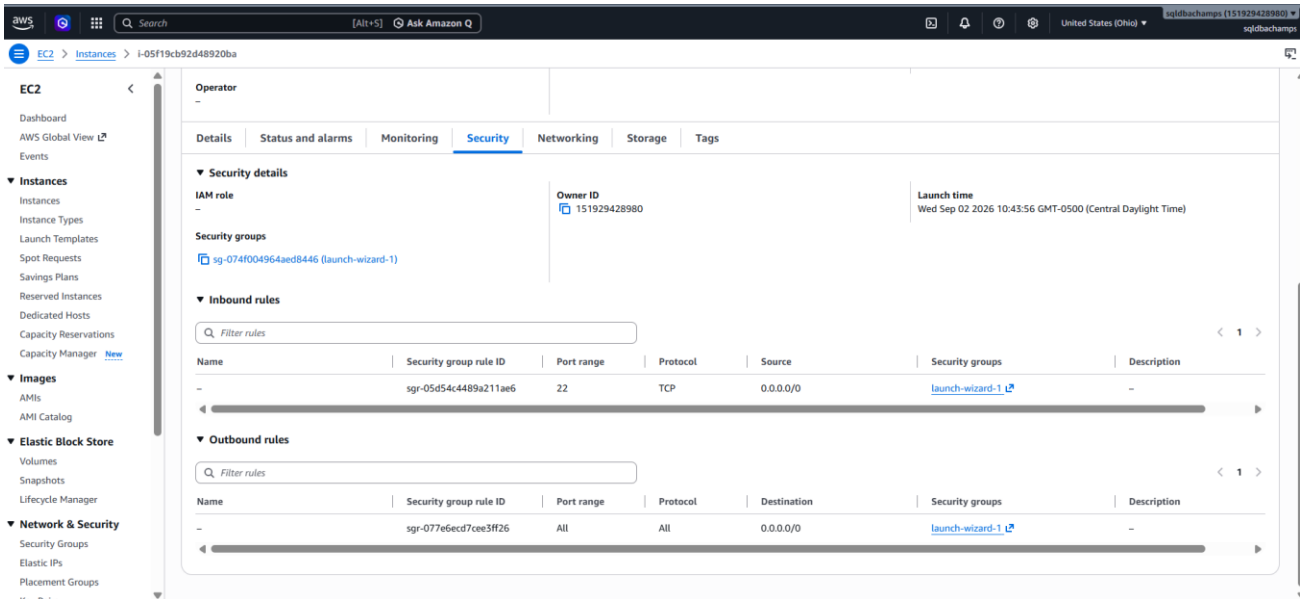


Figure B-7 - AWS EC2 Security tab showing the final security rule context.

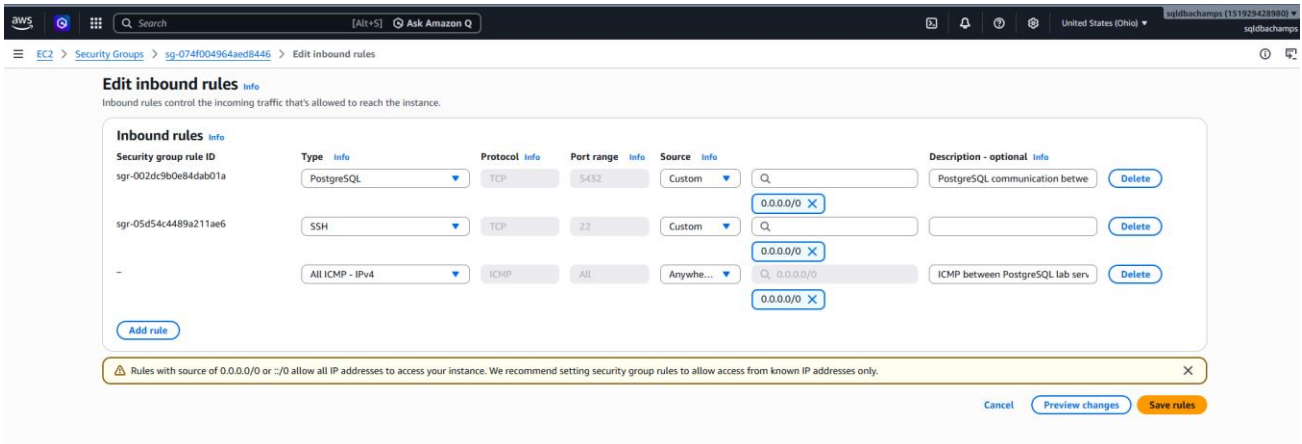


Figure B-8 - AWS Edit inbound rules screen used to configure required lab access.

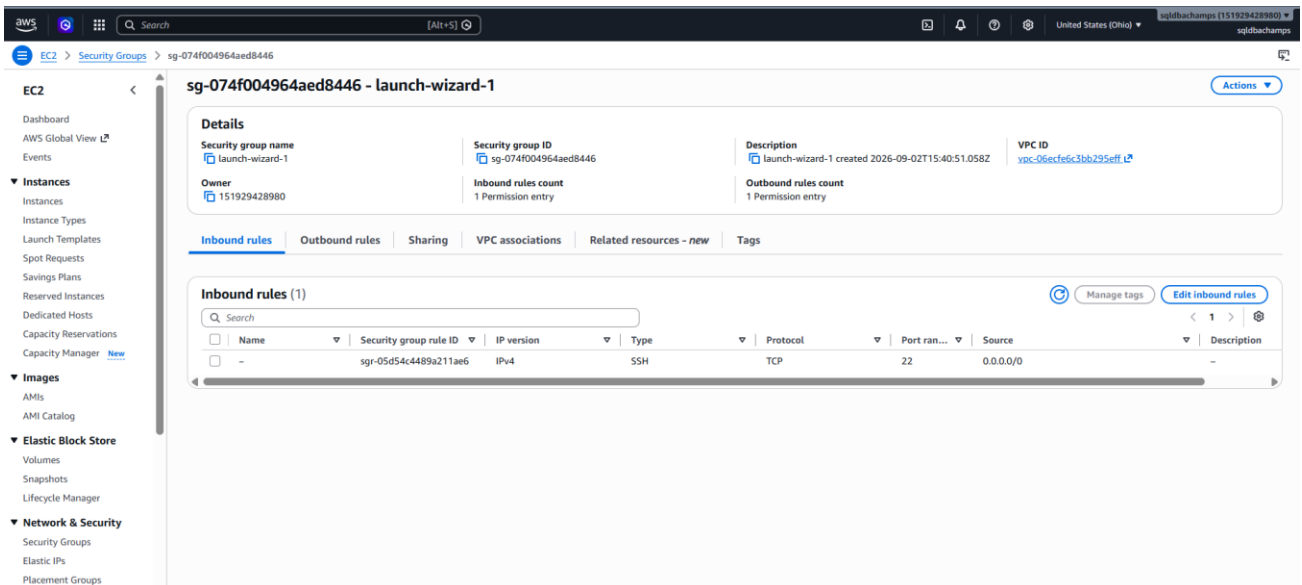


Figure B-10 - AWS security-group inbound rules after configuration.

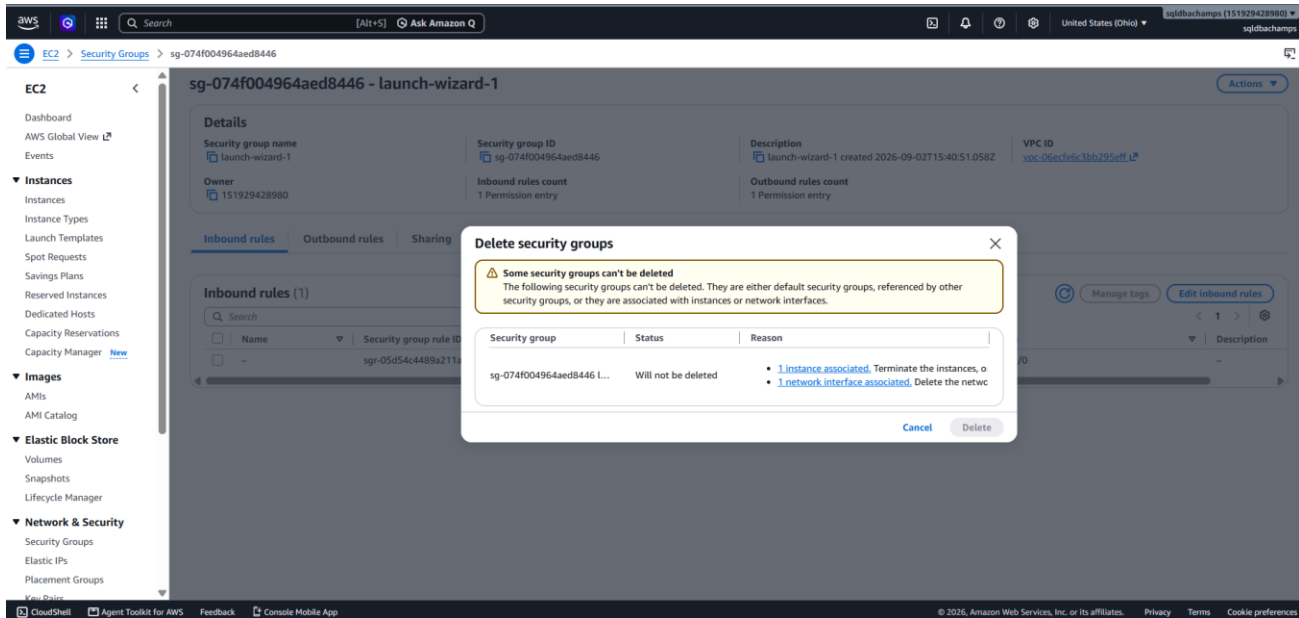


Figure B-11 - AWS security-group assignment/change screen captured during lab setup.

```
[ec2-user@db1 ~]$ sudo -u postgres psql -x -c "
SELECT name, setting
FROM pg_settings
WHERE name IN
(
    'wal_level',
    'max_wal_senders',
    'max_replication_slots',
    'hot_standby',
    'wal_keep_size'
)
ORDER BY name;
"
-[ RECORD 1 ]-----
name      | hot_standby
setting   | on
-[ RECORD 2 ]-----
name      | max_replication_slots
setting   | 10
-[ RECORD 3 ]-----
name      | max_wal_senders
setting   | 10
-[ RECORD 4 ]-----
name      | wal_keep_size
setting   | 0
-[ RECORD 5 ]-----
name      | wal_level
setting   | replica
[ec2-user@db1 ~]$ |
```

Figure B-12 - DB1 PostgreSQL replication prerequisite parameter validation.

```

labdb=# SELECT * FROM replication_validation;
 id | message | created_at
-----+-----+-----
  1 | Inserted on DB1 Primary | 2026-09-02 17:23:38.54257
6+00
  2 | PostgreSQL 18.6 Streaming Replication Test | 2026-09-02 17:23:38.54257
6+00
(2 rows)

labdb=# \q
[ec2-user@db1 ~]$ sudo -u postgres psql -c "
SELECT
    application_name,
    client_addr,
    state,
    sync_state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(
        pg_wal_lsn_diff(sent_lsn, replay_lsn)
    ) AS replay_lag_bytes
FROM pg_stat_replication;
"
 application_name | client_addr | state | sync_state | sent_lsn | writ
e_lsn | flush_lsn | replay_lsn | replay_lag_bytes
-----+-----+-----+-----+-----+-----
 walreceiver      | 172.30.2.128 | streaming | async      | 0/4039E80 | 0/40
39E80 | 0/4039E80 | 0/4039E80 | 0 bytes
(1 row)

[ec2-user@db1 ~]$

```

Figure B-13 - DB1 validation data plus pg_stat_replication output showing streaming replication.

```

[ec2-user@db1 ~]$ sudo -u postgres psql -c "
SELECT
    client_addr AS standby_ip,
    state,
    sync_state AS mode,
    pg_size_pretty(
        pg_wal_lsn_diff(sent_lsn, replay_lsn)
    ) AS replication_lag
FROM pg_stat_replication;
"
 standby_ip | state | mode | replication_lag
-----+-----+-----+-----
 172.30.2.128 | streaming | async | 0 bytes
(1 row)

[ec2-user@db1 ~]$

```

Figure B-14 - DB1 compact health check: DB2 at 172.30.2.128, state streaming, mode async, lag 0 bytes.

B.1 Screenshot Interpretation

The DB1 health screenshot is the final primary-side confirmation of the completed lab: client_addr 172.30.2.128, state streaming, sync_state async, and replication lag 0 bytes at the time of capture. Only successful and useful validation screenshots are included in this SOP. On DB2, pg_stat_wal_receiver and pg_is_in_recovery() are the appropriate standby-side health checks.